COMP 426 Assignment 2: 2D Cell Growth Simulation with Intel TBB

Charles-Philippe La Richeliere, 40113033

October 31st 2026

Overview and assumptions

This program simulates cell growth on a grid of 1024 by 768 cells. Each cell is healthy or cancerous, and groups of medicine cells are injected at several points. At every time step three rules are applied: a healthy cell with at least six cancerous neighbours becomes cancerous, a cancerous cell with at least six medicine neighbours is cured and uses up the medicine around it, and every remaining medicine cell moves one cell outward from its injection point.

The program is written in C++17. The parallel computation uses Intel oneTBB 2021.11 and the display uses OpenGL with freeglut, one pixel per cell: green for healthy, red for cancerous, yellow for medicine.

There is one control thread, the main thread of the program, and a pool of TBB worker threads, one per logical processor by default. The number of workers can be set with the `-t` option. Thirty times per second the control thread runs one time step. As required by the handout, each injection is simulated as a parallel task that moves its medicine cells at the beginning of the step; when all injection tasks have finished, the cell growth rules are applied to the whole grid in parallel. The control thread then displays the grid and reports the throughput.

Assumptions

1. The medicine cells of an injection form a filled disc around the injection point. The cells under the disc start as healthy cells.
2. A medicine cell passing over a healthy or cancerous cell does not change it. A cancerous cell is only cured by rule 2, with at least six medicine neighbours.
3. Cells outside the grid are not neighbours. A medicine cell that leaves the grid is removed.
4. One computation is one cell evaluated by the rules or one medicine cell moved.

5. Because medicine moves before the rules are applied, the medicine used up by a cure in step t is removed at the beginning of step $t+1$, before any medicine moves.

Architecture

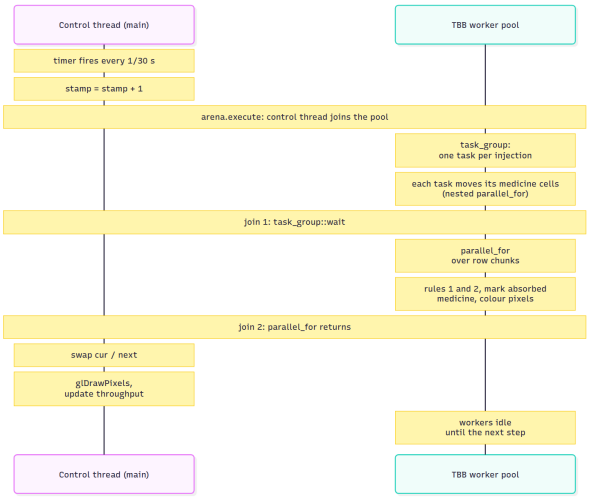


Figure 1: One step of the simulation. The two bars are the joins: `task_group::wait` and the end of `parallel_for`. Between them the TBB workers run in parallel and the control thread works as one of them.

Modules

Module	Role	Threading
`grid`	the cell grid, the medicine layer, the absorb map and the pixel buffer	none
`rules`	rule 1 and rule 2 applied to a band of rows; colours the pixels of those rows	none

Module	Role	Threading
`medicine`	one injection: the initial disc and the move of a range of its medicine cells (rule 3)	none
`sim`	one time step as TBB tasks, the thread pool and the throughput counters	the only module that includes TBB
`render`	the window, the 1/30 s timer and the OpenGL drawing; **this is the control thread**	freeglut main thread
`main`	reads the thread count and the injections from the command line	—

`sim` replaces the `threads` module of Assignment 1. The simulation logic in `grid`, `rules` and `medicine` contains no threading and no graphics, as in Assignment 1, so the same functions are called by the TBB tasks, by the single-threaded tests and by the benchmark. What changed is *who* calls them. In Assignment 1 one pthread owned a band of rows and one injection, because a thread was the only unit of work available. Here the rows and the injections are two different kinds of TBB work: the injections are tasks in a `task_group`, and the rows are the range of a `parallel_for`, split and balanced by TBB over whatever number of workers exists.

Data structures

All grids are 768 by 1024 static arrays in `grid.cpp`, reached through pointers so that the current and next cell grids can be exchanged by swapping two pointers.

Array	Type	An element means	Written by	Read by
`cur`	`uint8_t`	the cell is HEALTHY or CANCER at the start of the step	nobody during a step	rules

Array	Type	An element means	Written by	Read by
`next`	`uint8_t`	the cell's value after the step	rules, own rows only	becomes `cur` at the swap
`med`	`uint32_t`	`== stamp`: medicine is here in this step	injection tasks	rules, display
`absorb`	`uint32_t`	`== stamp`: a cure used up this cell in this step	rules	injection tasks, next step
`rgb`	`uint8_t[3]`	the pixel colour	rules, own rows only	OpenGL

Stamps instead of clearing

In Assignment 1 the medicine layer and the absorb map were byte arrays that had to be cleared every step, which cost a pass over the grid and a barrier so that no thread placed medicine in a row another thread was still clearing. Here both arrays hold a step number. A global counter `stamp` is incremented once per step; a cell holds medicine if `med[y][x] == stamp`, and was absorbed if `absorb[y][x] == stamp`. An old value simply stops matching, so nothing is ever cleared and the medicine layer needs no second copy. With 32-bit stamps the counter wraps after 4.5 years at 30 steps per second.

One consequence must be respected: when a cell is absorbed in two consecutive steps, the rules of step *t+1* overwrite the stamp of step *t*. The injection tasks therefore read `absorb` against the previous stamp *before* the rules run, which is the order of the step anyway.

Medicine cells

```
struct particle { float x, y; float dx, dy; bool alive; };

struct injection {
  int cx, cy;
  std::vector<particle> p;
  void init(int cx, int cy, int n);
  void move(size_t i0, size_t i1, uint32_t prev);
  int alive() const;
};
```

A particle stores its position and a unit direction away from the injection centre, as in Assignment 1. `move` works on a range of particles rather than the whole injection, so that one large injection can be split into chunks by a nested `parallel_for`. Injections are kept in a `std::vector` in `sim`; there is no fixed maximum because the number of injections is no longer tied to a number of threads.

The computation: TBB tasks

One step

The control thread calls `sim::step` thirty times per second. The whole step runs inside `arena.execute`, so the control thread becomes one of the workers of the arena for the duration of the step.

```
arena->execute([] {
  uint32_t prev = stamp;
  stamp = prev + 1;
  tbb::task_group tg;
  inject_all(tg, prev);   // one task per injection
  tg.wait();              // join 1: all medicine is in place
  rules_all();            // parallel_for over rows; join 2 when it returns
});
grid_swap();
```

Phase A, the injections. `inject_all` calls `task_group::run` once per injection. Each task moves the medicine cells of its own injection: a cell absorbed in the previous step or leaving the grid is removed, every other cell advances one position and stamps its new cell in `med`. Inside the task the particles are split into chunks of 1024 with a nested `parallel_for`, so one injection of 5000 cells does not become the critical path while the small ones are already finished. `tg.wait()` returns when every task is done.

Phase B, the rules. `rules_all` is a single `parallel_for` over the range of rows 0 to 768 with a grain of 8 rows. TBB splits the range recursively and idle workers steal halves, so the rows are balanced over all cores whatever the number of injections. Each chunk reads `cur` and `med`, writes its own rows of `next` and `rgb`, and stamps `absorb` around every cured cell. The `parallel_for` returns when every chunk is done; that return is the second join of the step.

TBB construct	Replaces, from Assignment 1
`task_arena(n)`, `execute`	`pthread_create` of n threads; `-t n` sets the arena size
`task_group::run` / `wait`	one thread per injection; `wait` is the barrier
`parallel_for(blocked_range)`	the fixed band of rows per thread
`affinity_partitioner`	nothing; the same rows go to the same worker every step, so they are still in that core's cache
`enumerable_thread_specific`	the `count` field of each worker
`tick_count`	`clock_gettime`

Why there are no locks

A lock is needed when two tasks can touch the same memory at the same time and at least one writes. The step is arranged so that this never happens with a

value that matters.

- **Shared reads.** During a step `cur` is never written, and `med` is only written in phase A and only read in phase B. Any number of tasks may read them.
- **Disjoint writes.** A rules chunk writes only its own rows of `next` and `rgb`. Two chunks never share a row.
- **Same-value writes.** Two cases remain where two tasks can write the same cell in the same phase: medicine from two injections moving onto the same cell, and two cured cells that have a neighbour in common. In both cases every writer stores the same value, the current stamp, so the result does not depend on the order. The store is a relaxed atomic store so that it is a single, well-defined write.
- **Private counters.** Each worker counts its computations in its own slot of an `enumerable_thread_specific`; the slots are only summed by the control thread between steps.

The only synchronisation in a step is therefore the two joins, `tg.wait()` and the return of `parallel_for`. There is no mutex, no condition variable and no barrier object. Compared with Assignment 1, which had three barrier waits per step, the clearing of the medicine layer no longer needs its own phase.

The control thread

The control thread is the main thread of the program. As in Assignment 1 it cannot run a loop of its own: freeglut owns the main thread once `glutMainLoop` is called, and all OpenGL calls must come from the thread that created the window. One step is therefore the body of a timer callback.

```
static void tick(int) {
  glutTimerFunc(33, tick, 0);   // re-arm first, so the period stays 33 ms
  step_fn();                    // sim::step
  glutPostRedisplay();
}
```

`render_run` receives the step function and the status function as pointers, so the render module depends only on `grid` and can be tested with a dummy step. Inside `sim::step` the control thread enters the arena and executes tasks like any worker; when `execute` returns, every task of the step is finished and the grids are swapped. The redraw copies `rgb` to the window with `glDrawPixels`, one pixel per cell, and swaps the buffers.

Throughput. One computation is one cell evaluated by the rules or one medicine cell processed. Every chunk adds its count to the calling worker's slot of an `enumerable_thread_specific`; the step time is measured with `tick_count` around `execute`. Once per second the control thread sums the slots and divides by the accumulated step time. It does this between steps, when no task is running, so the sum needs no synchronisation.

Thread count. `sim::init(n)` creates a `task_arena` with `n` slots; with no `-t` option it uses `max_concurrency()`, the number of logical processors of the machine. The status line shows the arena size, the number of injections, the throughput and the mean step time.

Optimisations

The aim is the same as in Assignment 1: keep every worker on memory that is its own, and keep the cost of synchronisation independent of the size of the grid.

- **Stamps, no clearing.** The medicine layer and the absorb map are never cleared and never copied. This removes one pass over 1.5 MB per step and the barrier that protected it.
- **Pixels coloured by the rules.** The rules pass already has the new cell value and the medicine flag in registers, so it writes the pixel directly. The separate `fill_rgb` pass of Assignment 1, 786 432 cells per step on the control thread, is gone.
- **Clamped neighbour box.** The bounds of the 3 by 3 box are computed once per cell, four comparisons instead of eighteen.
- **Affinity partitioner.** The row chunks go to the same worker every step, so a worker's rows are usually still in its cache. This addresses the cold cache problem measured in Assignment 1.
- **Nested `parallel_for`.** A large injection is split into chunks, so the duration of phase A is bounded by the chunk size, not by the largest injection.
- **Two joins per step, no locks**, as described above.
- **Pointer swap**, one byte per cell, persistent workers, as in Assignment 1.

Results

Measured with `make bench`: 600 steps back to back, no window, 16 injections of 1000 cells, on an Intel Core i9-12900KF (8 performance cores, 8 efficiency cores, 24 logical processors), `g++ -O2`. The Assignment 1 column is the pthread version from the same machine, with one injection per thread.

Threads	A2, million computations/s	Speed-up	ms per step	A1, million computations/s
1	152.7	1.0	5.25	59.1
2	299.7	2.0	2.68	115.9
4	594.5	3.9	1.35	224.2
8	1138.8	7.5	0.70	379.2
16	1645.5	10.8	0.49	593.0
24	1769.3	11.6	0.45	—

The single-thread throughput is 2.6 times that of Assignment 1; that gain is the

stamps, the merged pixel pass and the clamped neighbour box, not parallelism. The speed-up is near linear up to 8 threads, the number of performance cores, and flattens beyond because the remaining logical processors are efficiency cores and hyperthreads sharing the same memory bandwidth. At 24 threads a step takes 0.45 ms, far below the 33 ms period, so the program at 30 steps per second is limited by the display, not the computation.

Test scenarios

The visual scenarios are run with `./demo.sh N` and show the behaviour of the whole program. The automated tests are `make` targets and check what cannot be judged by eye at one pixel per cell.

Visual scenarios

N	Setup	What it tests	Expected result
1	four injections, all cores	several injection tasks at once	four rings, cured areas behind them, status line
2	same four, `-t 1`	the result does not depend on the arena size	same picture as 1, higher ms per step
3	300, 1500 and 5000 cells	the nested `parallel_for`: one chunk vs several	discs of radius 10, 22 and 40, cured areas in proportion
4	two injections 60 cells apart	two tasks writing the same `med` cells	the rings pass through each other, no medicine left behind
5	injections in the four corners	the edges of the grid in `move` and in the neighbour count	no crash, medicine disappears at the edge
6	sixteen injections, `-t 4`	more tasks than threads	sixteen rings, same behaviour, TBB queues the tasks

Automated tests

`make test` builds small hand-made grids and checks the rules at their boundaries, then compares a multi-threaded run with a single-threaded one.

Test	What it proves
rule 1	6 cancerous neighbours mutate a healthy cell, 5 do not
rule 2	6 medicine neighbours cure a cancerous cell, 5 do not

Test	What it proves
absorb	the medicine around a cured cell is gone at the next step; medicine around an uncured cell stays
move	after 10 steps a medicine cell is 10 cells further out, none is lost, the layer shows every live cell
edge	medicine that leaves the grid is removed, no yellow remains
threads	after 100 steps, all threads give the same checksum as 1 thread

`make ordertest` is the test for the ordering of a step. Sixteen injections are packed around the centre so that medicine from different tasks shares cells from the first step. For 300 steps, after every step, the control thread checks against the saved previous state that every medicine cell was used up, removed at the edge or moved exactly one cell; that the medicine layer holds exactly the live cells; and that every cell has the value rule 1 and rule 2 give for the old grid and the new medicine positions. The final checksum must be identical on every run.

Version of the program	Result
final design	all three checks pass, checksum `1ec1bf255fc35263` on every run
`tg.wait()` removed (`make brokentest`)	medicine checks pass, about 1000 cells wrong per run, different checksum each run

The last row is the experiment that shows what the join is for. With the join removed the injection tasks are still correct on their own, but rule chunks run while medicine is still being placed, so some cancerous cells are evaluated against an incomplete medicine layer.

The most difficult part